FIT2099 - Assignment 2 Design Rationale

Jingyu Zhang 37033956 – REQ 1: Harvesting under the moon

REQ1 asks for three things: harvesting, watering and eating crops. This rationale follows those three. For each one, we say what Assignment 1 had, what we changed, and why the change keeps the code easy to extend.

A. Harvesting a crop

Assignment 1: Harvestable had two methods, getHarvestStaminaCost() and harvest(). One generic HarvestAction found it with getGroundAs. We kept this design, because a new crop only implements Harvestable and the action does not change.

Rejected: One HarvestXxxAction per crop.

Pros	Cons
Independence: Each action holds all the rules of its crop, so a reader finds them in one class.	DRY: Every class copies the same protocol: is the crop ready, and can the actor pay the stamina?
KISS: There is no interface to learn.	OCP: A change such as Cauliflower's five waterings needs the same edit in every class.

What changed, and why. First, Harvestable gained two default methods, canHarvest() and cannotHarvestMessage(). Cauliflower is harvestable only after five waterings. Without the hooks, the action would need an if on the crop type. Now only Cauliflower overrides the defaults. Second, Harvestable and the other capability interfaces moved to game.actions, next to the actions that use them. So crops depends on actions, and there is no package cycle. Third, the stamina check moved from a static StatisticUtils to StatisticAdjuster. trySpend checks and spends in one step, so every crop's harvest() can assume the actor already paid. Fourth, Assignment 1 put tile effects in a static LocationUtils. Parsnip drops a bonus item when harvested, Strawberry drops items when watered, and Melon may explode when harvested. The static class grew into unrelated helpers, and each one rebuilt the list of nearby tiles. We replaced it with Area (a group of tiles), ItemDropper (places items on an Area) and Explosion (hurts every actor in an Area). Area.filter takes TileCondition objects, so we use no lambdas.

Pros	Cons
SRP/OCP: HarvestAction owns the shared protocol. Each crop owns only its own effect. A new crop implements Harvestable, and nothing else changes.	A crop's harvest effect is a method on the crop. So the crop class knows about inventories and ground replacement, as well as its own rules.
DRY: HarvestAction checks and spends stamina once. The "adjacent" radius and the tile-picking loop also exist once, in Area.	Crops must not spend stamina themselves. We document this on the interface, but the compiler does not enforce it.
OCP: A new area shape, effect or condition is a new method or class. No existing one changes.	ItemDropper is a thin wrapper around an Area, and a condition is a class even when it is one line long.
KISS: Harvesting needs one protocol, one action	

and one interface. An effect reads as one Area call and one action.	

Connascence: The rejected design tied each crop to its action by name and position. Now they share only one interface (Connascence of Type). The radius "1" was repeated in every helper (Connascence of Value). Now it is one constant. Foresight: A Pumpkin that needs two waterings overrides canHarvest() and cannotHarvestMessage(), and HarvestAction stays the same. A Frost effect on the tiles within two of a crop is Area.within(centre, 2) plus one new effect class.

B. Watering a crop

Assignment 1: Waterable and WaterAction watered a ground, so Dirt became a Puddle. TillAction and WaterAction each checked the tool, the capability and the stamina, and each kept its own stamina cost. REQ1 adds crops that can be watered. A crop's watering has a count, its own stamina cost and its own effect.

Rejected: Two interfaces and two actions: Waterable with WaterAction for the ground, and CropWaterable with WaterCropAction for crops.

Pros	Cons
SRP: Dirt only says what it becomes. The action changes the map and records the watering.	DRY: The two actions have the same structure. They differ only in the effect and the cost.
ISP: Each interface is small and has one kind of client.	OCP: A third waterable kind needs a third action.

Chosen: One Waterable interface and one WaterAction. Waterable has water(actor, location), which applies the effect, and a default method getWateringStaminaCost(toolCost), which returns the tool's cost. WaterAction extends CapabilityToolAction<Waterable, WateringTool>. This template holds the checks that Assignment 1 copied into each action, and TillAction shares it. The stamina cost moved from a constant in each action to the tool (HasStaminaCost). WaterAction only asks the ground for its cost and calls water(). Dirt turns itself into a Puddle. Each crop records the watering and then applies its own effect. The four crops override the cost, because a crop has its own.

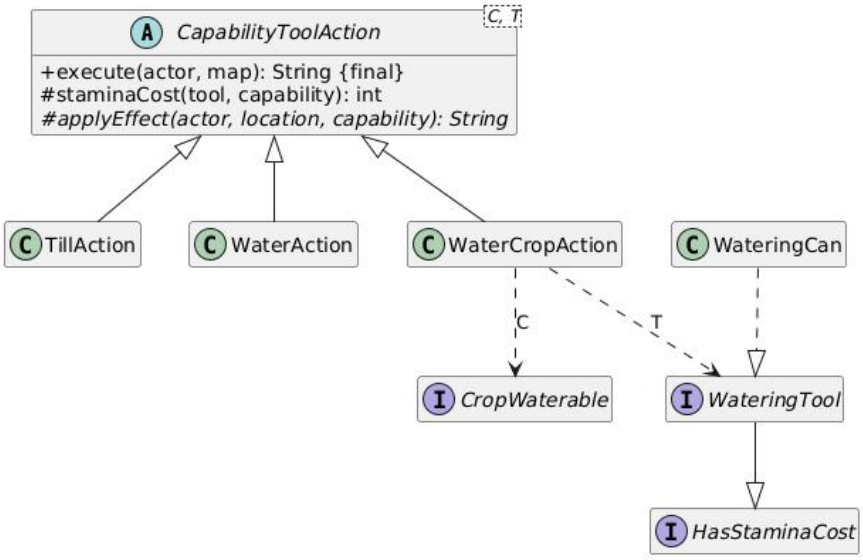


Figure 1: the template that watering shares with other tool actions.

Pros	Cons
DRY: One action serves every waterable thing. No two classes have the same structure.	Connascence of Execution: Each crop's water() must call recordWatering() first. The action used to guarantee this. Now four crops must remember it. We write the rule on the interface.
OCP: A new waterable thing only implements Waterable. The action does not change.	SRP: Dirt now changes the map itself. Before, it only said what it became, and the action changed the map.
DIP: The action depends on one abstraction. The ground or the crop decides what watering does.	

Crop effects. Cauliflower turns the nearby ground that holds no crop into puddles. It uses Area.filter(new Unplanted()), a TileCondition in game.crops, because area must not depend on crops. Parsnip, Cauliflower and Melon roll a chance. Assignment 1 gave each class its own Random. We now use RandomUtils, whose static methods pass the work to one replaceable instance. Every roll goes through one point, so a run can be repeated. Parsnip's chance: The brief says "10% plus the number of times it has been watered". We count the watering in progress, so the first watering is 11%. Parsnip.water() records the watering before it rolls. The tutor confirmed this reading.

Connascence: The action and its base agree on two hooks (Connascence of Type), and the compiler checks them. The watering count lives in Crop, and a crop's effect reads it through getWateringCount(). Foresight: A Sapling only implements Waterable. A Fertiliser tool and a Fertilisable ground need only a new subclass of CapabilityToolAction. A "lucky day" that raises every chance by 10% is one subclass of RandomUtils that overrides doRollPercent.

C. Eating produce

Assignment 1: Consumable was a capability interface. An abstract ConsumableItem offered one generic ConsumeAction, and each item only implemented consume(). FoodItem extended it for items that also feed animals. We kept this design.

Rejected: One eat action per produce item.

Pros	Cons
Independence: Each item has its own action and its own menu text.	DRY: Every action repeats "remove the item, apply the effect".
KISS: There is no Consumable interface to learn.	OCP: A new edible item needs a new action, so the menu code grows with every item.

What changed, and why. First, consume() now reports the real change, such as "1 HP (99 -> 100)", not a fixed amount. Assignment 1 used a static StatisticUtils. We replaced it with StatisticAdjuster, an object bound to one actor. Its increase and increaseMaximum return a StatisticChange, a small value object that describes itself. So changing a statistic and writing the message are separate jobs. We rejected making increase return a string, because it would mix the two jobs. Second, ConsumableItem has two helpers, eatToHeal and eatToRaiseMaxHealth. Four items heal or raise maximum HP, and the helpers keep their code to one line. Third, a rotten strawberry poisons the actor, and a fresh one heals it over several turns. We use the engine's Status. An abstract TimedStatus holds the countdown once, and each effect only gives its duration and applyEffect(). Fourth, Consumable has a default method, canBeConsumedBy(actor), so an actor that lacks what an effect needs can refuse the item (REQ5). Maximum increases: Raising a maximum refills the current value. The engine's BaseStatistic.increaseMaximum already works this way, and the tutor confirmed it. Tomato and the feeding bonuses call increaseMaximumWithoutRestoring.

Pros	Cons
SRP/OCP/DRY: ConsumeAction only removes the item and applies consume(). A new edible item is one class with a one-line consume().	ConsumableItem exposes two protected helpers that subclasses must use correctly (Section E).
ISP/LSP: The three items that are only eaten never inherit feeding behaviour. Any Consumable can stand in for another in ConsumeAction.	The hierarchy has one extra class (FoodItem).
SRP/DRY: Each rule about statistics has one definition, and we change the message wording in one class. The countdown of timed effects exists once.	The code creates a short-lived object at each use. The rules sit beside the engine's Actor, not on it, because we cannot change the engine. applyEffect() is a protected contract that a reader must learn.
KISS: A new edible item is one class, and a new timed effect is one small subclass.	

Connascence: Every call site used to agree on the same two-step algorithm for statistics (Connascence of Algorithm). Now they agree only on method names. Foresight: A Pumpkin that heals 30 HP is return eatToHeal(actor, 30);. A Spicy Pepper that costs 2 stamina for 5 turns is one TimedStatus subclass. "An exhausted actor cannot spend stamina" is one change in trySpend.

D. Other changes to the Assignment 1 design

Characters: The new crop items use c, p and s. So we changed Cow C to B, Chicken c to n, and Milk M to K, and nothing else in the story. Packages: The new rules added many classes, so we split the

flat packages into sub-packages (crops, actions, positions.farm, items.food, status) without cycles. Names: ParsnipCrop became Parsnip, and the old item Parsnip became ParsnipItem (the same for Tomato). The Item suffix keeps a crop and its item apart.

E. Known minor violations and why we accept them

Violation or cost	Why it is acceptable
RandomUtils is a static facade. It holds a global mutable instance, and DIP is only partly met. Nothing uses the replaceable instance yet.	Without it, randomness has no single point of control. The alternative is to pass a random object through the constructors of about thirty-five classes. The engine creates several of them through suppliers. That costs far more than it returns.
StatisticAdjuster sits beside the engine's Actor, not on it.	The rules concern an Actor. But we cannot modify the engine. An object bound to one actor is the closest option.
static members: constants; the factories Area.adjacentTo, centredOn, within, wholeMap and ItemDropper.adjacentTo; StrawberryItem.createRandom(); and RandomUtils.	A plain constructor cannot say which tiles it means. createRandom() decides rotten or fresh before the constructor runs, because the item's name goes to super(...). Nothing else in REQ1 is static.
protected members: CapabilityToolAction.staminaCost and applyEffect, TimedStatus.applyEffect, ProcessedProduct.canReceiveProcessingEffect, the ConsumableItem helpers and RandomUtils.doRandomInt.	They are template-method hooks and helpers for subclasses only. If they were public, callers could skip the checks. No attribute is protected. Every instance field is private.
Crop creates a concrete Dirt, and Cauliflower creates a concrete Puddle.	Each has one possible outcome. An injected factory would add a parameter that nobody varies (YAGNI).
WaterAction keeps its own target for its menu text.	The base class's location is private. We prefer a second copy to a protected attribute.
We use Supplier factories (placeOnEveryTile).	Every tile needs its own new Item. This is the factory pattern, not functional style.

Summary

Each of the three requirements has one generic action and a small capability interface (Harvestable, Waterable, Consumable). So a new crop, tool or produce item is a new class (SRP, OCP, DIP). Any implementation can replace another (LSP), and each interface is small (ISP). Each rule exists once: the action protocols, StatisticAdjuster, Area and TimedStatus (DRY), with the simplest structure that does so (KISS). Connascence stays at Name and Type. Section E lists the few places where we bend a principle, with the reason.

Yuanshu Liu 36226173 – REQ 2: Mining in the Mines

REQ2 introduces three ways to travel between the farm and a new map, the mines, and three objects in the mines that must be broken rather than entered. The design goals were: keep each mechanism's own rule (a ladder's fall, a circle's fire, the alien device's malfunction; a rock's shockwave, a copper node's charge, a geode's explosion) inside the class that owns it; let a new travel mechanism or a new breakable object be added without editing the Action that dispatches it; and avoid repeating the same "find a tile around this location" search in every feature that happens to need one. Abstractions were only introduced where REQ2 itself already repeats behaviour across at least two of its own classes (DRY), not where a future requirement might one day want something similar (YAGNI).

A. Dispatching travel through a TravelMethod capability

Design 1: TravelAction is written once per mechanism, or holds an if / else on a mechanism type to decide whether to roll a fall, light a fire, or roll a malfunction.

Pros	Cons
No extra interface to read.	Every new mechanism (a future "magic carpet", which the spec explicitly anticipates) means editing TravelAction again, and the branches multiply as more mechanisms are added — an OCP problem.

Design 2 (chosen): TravelMethod is an interface with two default methods, chooseLanding (default: the intended destination) and onArrival (default: no effect). TravelAction depends only on this interface: it asks method.chooseLanding(...) , checks landing.canActorEnter(actor) (the engine's actor-location iterator throws if the tile is occupied, so this guard is required, not optional), calls map.moveActor(actor, landing) , then asks method.onArrival(...) . Each mechanism overrides only the one hook its own rule needs: Ladder overrides onArrival only (a fall never changes where the ladder leads); TeleportationCircle overrides onArrival only (the fire is an arrival effect, not a different landing spot); AlienDevice overrides chooseLanding only (a malfunction changes where the carrier ends up, but has no separate arrival effect).

Pros	Cons
TravelAction depends on the TravelMethod abstraction, not on Ladder / TeleportationCircle / AlienDevice (DIP). A new mechanism is one new class; TravelAction is never touched again (OCP). Because the two methods are optional defaults rather than compulsory overrides, each mechanism writes only the behaviour it actually varies in, instead of boilerplate for behaviour it does not have.	A reader has to open TravelMethod to learn the two hooks exist before a new mechanism's code makes sense on its own.

B. Sharing destinations between a Ground mechanism and a carried mechanism

A ladder and a teleportation circle are Ground (TravelGround extends Ground implements TravelMethod); the alien device is an Item the farmer carries (AlienDevice extends Item implements TravelMethod). Both still need the same bookkeeping: a name-to Location list, and one TravelAction built per entry.

Design 1: Give TravelGround and AlienDevice their own private destination map and their own loop that turns it into an ActionList .

Pros	Cons
No new class.	The destination bookkeeping and the loop that builds actions from it are written twice — a DRY violation, since both copies would need to change together if, for example, how destinations are ordered in the menu changed.

Design 2 (chosen): The bookkeeping is pulled into DestinationList , a small class with add(name, location) and actionsFor(TravelMethod method) . TravelGround and AlienDevice each hold one as a field and delegate to it. The two classes cannot share this logic through inheritance — TravelGround must extend the engine's Ground to be placed with setGround(...) , while AlienDevice must extend Item to be carried, and Java does not allow extending both — so composition here is not a stylistic preference but the only way to reuse the logic at all.

Pros	Cons
The destination list and the action-building loop exist exactly once, independently of which engine superclass the mechanism needs (DRY). DestinationList has exactly one job — it knows nothing about falls, fire, or malfunctions (SRP).	DestinationList is a second small class a reader must find, for logic that, on its own, is barely ten lines.

C. A Breakable capability shared by Ground and an Item

Rocks and copper nodes are *Ground*; a geode can be broken while still an *Item* in the inventory, or after being dropped on a tile. *Breakable* is the one method both shapes implement: *String breakUp(Actor actor, Location location)*. *BreakAction* only checks that a *BreakingTool* the actor carries can break the target (*tool.canBreak(target)*, found through *getItemsAs(BreakingTool.class)*) and then calls *target.breakUp(...)* — it never asks what *target* actually is.

BreakableGround (abstract, implements Breakable , canActorEnter always false) supplies a final breakUp() that turns the tile to Dirt , drops the item the subclass supplies through createDrop() , and calls the subclass's afterBreaking() for the rest of the effect — a template method, since "turn to dirt, then drop something, then do one more thing" is identical for Rock and CopperNode and only the "something" and the "one more thing" differ. Geode cannot reuse this template, since it is an Item , not a Ground ; it implements Breakable directly and writes its own short removal-from-inventory-or-tile step before the diamond/explosion roll.

Why *breakUp()* is *final*. The spec says every broken rock-like object turns into dirt first. If a subclass could override *breakUp()*, a new rock-like ground could forget that rule. So *breakUp()* is *final*, and a subclass can change only the two things that really differ: the item it drops (*createDrop()*) and the extra effect (*afterBreaking()*). The cost is that a subclass cannot reorder the steps. No requirement needs that, so we accept it.

Design 1: One independent breakUp() per concrete class (Rock , CopperNode , Geode).

Pros	Cons

No abstract base class, no template method to learn.	Rock and CopperNode would repeat "become Dirt, drop an item, then…" verbatim (DRY), and nothing would stop a later ground based breakable from forgetting the "becomes Dirt" rule the spec states for every broken object.

Design 2 (chosen): Breakable as the one dependency BreakAction holds; BreakableGround 's template method for the two Ground cases; Geode implementing Breakable on its own because it is not Ground-shaped.

Pros	**Cons**
BreakAction depends only on Breakable (DIP) — a future breakable Item such as a treasure chest, or a future breaking tool such as a hammer (both explicitly anticipated by the spec), need no change to BreakAction , BreakableGround , or each other (OCP). The "becomes Dirt, then drop, then effect" rule is written once, in BreakableGround , and cannot be forgotten by a new rock-like ground (DRY).	createDrop() and afterBreaking() are declared protected , though Rock and CopperNode are the only subclasses today and both already sit in game.positions.mine , the same package as BreakableGround — package-private visibility would express the same capability without exposing the hooks outside the package. This is a minor compromise, not a defect: no other code calls these two methods, and tightening the modifier later costs nothing.

D. Why breaking does not reuse CapabilityToolAction

REQ1's tilling and watering already share CapabilityToolAction<C, T extends HasStaminaCost> , an abstract Action that checks a tool in the inventory, a matching capability on the Ground at a Location (via getGroundAs(capabilityType)), deducts stamina, and delegates the rest to a subclass. TillingTool and WateringTool both extend HasStaminaCost to satisfy the generic bound.

Design 1: Make BreakingTool extends HasStaminaCost and write BreakAction as a third CapabilityToolAction subclass, for consistency with TillAction / WaterAction .

Pros	**Cons**
One fewer pattern in the codebase; breaking would look exactly like tilling and watering.	CapabilityToolAction looks up its capability only on the Ground at a Location ; it has no path to a capability an Item itself exposes, so it could never dispatch to Geode . The spec gives breaking no stamina cost, so Pickaxe would be forced to implement getStaminaCost() purely to satisfy a type parameter it never uses — the interface bloat ISP warns against — and inventing a cost concept the spec never asked for is the speculative generality YAGNI rules out.

Design 2 (chosen): *BreakAction* stays its own small *Action*. *BreakingTool* has one method, *canBreak(Breakable target)*: the tool says which targets it can break. *Pickaxe* answers true for every target. *BreakAction* asks each tool the actor carries and never names *Pickaxe*. *BreakingTool* does not extend *HasStaminaCost*, because the spec gives breaking no stamina cost.

Pros	**Cons**

OCP and LSP: A hammer that cannot split a geode is one new class that returns false for it, and *BreakAction* does not change. Neither *Breakable* target has to be *Ground*-shaped, so *Geode* is reached the same way as *Rock* and *CopperNode*. **ISP and YAGNI**: *BreakingTool* has one real method, and no stamina-cost concept is invented for a feature the spec is silent on.	Two sibling patterns now exist in *game.actions*: *CapabilityToolAction* for tilling and watering, and a plain *Action* for breaking. A reader comparing them must notice why they differ, rather than assume breaking simply "is" a *CapabilityToolAction*. *BreakingTool* also depends on *Breakable*, one more link between *items.tools* and *positions*.

E. One shared notion of “tiles around a location”, instead of five separate searches

Five of REQ2’s six new behaviours need to find a set of tiles and do something with them: AlienDevice picks a random enterable tile anywhere on the current map; TeleportationCircle drops a Fire on every tile adjacent to the landing tile; Rock picks a random enterable tile adjacent to itself to throw the farmer onto; CopperNode visits every adjacent tile looking for a Chargeable ground; Geode damages the centre tile and its neighbours, then sets fire to the neighbours only.

Design 1: Each class writes its own loop over the engine’s nearby-locations method (or, for the alien device, over every tile of the map), with its own random pick and its own “is this tile usable” check inlined.

Pros	**Cons**
No extra classes between a feature and the engine’s Location .	The same “collect tiles, keep only the ones that satisfy a condition, pick one at random” shape is written five times, with small, easy-to-miss variations (adjacent vs. whole-map; filtered by enterability vs. not filtered at all) — a DRY and cohesion problem, since a bug in one copy would not automatically be fixed in the other four.

Design 2 (chosen): Area is a small value object over a list of tiles, built from static factories (adjacentTo , centredOn , wholeMap) naming the shapes REQ2 actually needs, plus filter(TileCondition) and randomTile() / randomEnterableTile(actor) . TileCondition is a one-method interface (EnterableBy is its REQ2 implementation); *what is done* with the chosen tiles lives in two further single-purpose collaborators — ItemDropper (places or removes Item s on an area’s tiles, used by TeleportationCircle and Geode to lay down Fire) and Explosion (hurts whichever Actor stands on each tile, used by Geode). AlienDevice.chooseLanding and Rock.afterBreaking both call randomEnterableTile , so the “pick a random tile the actor can actually stand on” rule exists exactly once and is shared by two otherwise unrelated features.

Pros	**Cons**

Area only ever answers “which tiles, and which of them match a condition” — it never changes game state itself, so it cannot be the cause of a wrong damage number or a misplaced item (SRP). ItemDropper only places/removes items and Explosion only damages actors, so a change to how explosion damage is calculated cannot accidentally affect how tiles are chosen, and vice versa. A new way of narrowing an area is a new TileCondition implementation, never a change to Area itself (OCP). The randomness and the enterability check behind both the alien device’s malfunction and the rock’s shockwave come from one method, so a correction to either applies to both automatically (DRY).	Three small classes (Area , ItemDropper , Explosion) plus the TileCondition interface replace what could have been one method call per feature; a reader has to learn this small vocabulary before any one of the five call sites is self explanatory, and Area.filter walks its tile list once per call rather than short-circuiting — irrelevant at REQ2’s map sizes but worth revisiting for a much larger map.

F. A charged ground state, and reusing REQ1’s timed-status shape

Breaking a CopperNode must turn every adjacent Chargeable ground into its charged form; in REQ2 the only Chargeable ground is Puddle .

Design 1: A separate ChargedPuddle extends Ground class, returned by Puddle.getChargedResult() .

Pros	**Cons**
The charged and uncharged states are two unmistakably separate classes.	ChargedPuddle would duplicate everything about Puddle except its display character and its tick() body — a second class for a one-line behavioural difference.

Design 2 (chosen): Puddle keeps a private boolean charged field, set through a private constructor; the public no-argument constructor always creates an uncharged puddle, and getChargedResult() returns a new charged Puddle . tick(Location) shocks the occupant for 20 damage only when the puddle is charged and an actor is standing on it.

Pros	**Cons**
One class instead of two, for a ground that genuinely has only two states and whose entire behavioural difference is a single condition inside tick() (KISS) — a ChargedPuddle class today would be building for a third charge level the spec does not describe (YAGNI).	Compared to Design 1, this is a small, honest step away from OCP: if a later requirement introduced more than two charge levels, tick() ’s one condition would have to grow into a chain rather than simply gaining a new subclass. With exactly two states today, this is judged an acceptable trade rather than a defect.

Separately, *BurningStatus* (the burn a *Fire* inflicts) extends the same abstract *TimedStatus* that REQ1's *StrawberryPoisonStatus* extends. *TimedStatus* counts down a fixed number of turns and applies one subclass-specific effect each turn. We extracted it so that REQ1 and REQ2 share one

countdown, and a new kind of timed effect is one new subclass (DRY, OCP). *BurningStatus*'s only public constructor path first removes any burn the actor already carries before adding the new one. This is why a *Fire*'s tile reapplies its burn at full strength every turn an actor stays on it, and does not stack into an ever-growing damage total that the spec never asked for.

G. Display characters of three mine items

The map shows one character per tile, so two items with the same character look the same to the player. Three REQ2 items shared a character with a REQ1 crop item: Stone and Strawberry (both *s*), Pickaxe and Parsnip (both *p*), and Copper ore and Cauliflower (both *c*). We changed the three mine items: Stone is now *e*, Pickaxe is *x* and Copper ore is *u*. The mine items are new in Assignment 2, so changing them touches no Assignment 1 class and no crop. The change is one character in each constructor.

Pros	**Cons**
Low cost: One constant per class. No other class, message or diagram label changes.	**Hand-picked**: A later item could clash again. Someone must check the other characters first.
KISS: The map stays readable without a new class. A central registry of display characters would stop a future clash, but it would add a class for a one-character choice (YAGNI).	**Less memorable**: *e*, *x* and *u* are not the initials of Stone, Pickaxe and Copper ore.

H. Connascence and foresight

Connascence. *TravelAction* and a mechanism agree on two hook names and their types (Connascence of Name and of Type), and the compiler checks both. *Rock* and *CopperNode* share the order "turn to Dirt, drop an item, then apply one more effect" (Connascence of Execution). We keep this order in one place, *BreakableGround.breakUp*. The search for tiles around a location was written five times (Connascence of Algorithm). Now it exists once, in *Area*. **Foresight.** A magic carpet is one new class that overrides *chooseLanding*. A hammer is one new class that implements *BreakingTool*. A treasure chest is one new class that implements *Breakable*. A third charge level for the puddle means one change in *Puddle.tick*, which Section F accepts on purpose.

I. Changes to the Assignment 1 design

Change from Assignment 1	**Why we made it**
Puddle gained a charged state and implements *Chargeable*.	A copper node charges nearby ground. One class with one flag is simpler than a separate *ChargedPuddle* (Section F).
StackQueueValleyWorld adds the mines map, the ladders, the circles and the alien device. The Farmer now starts with a *Pickaxe*.	REQ2 needs a second map to travel to and a tool to break things. Only the set-up code changes.
Stone, *Pickaxe* and *Copper ore* use new display characters.	They clashed with REQ1's crop items (Section G).

Summary

REQ2's design rests on four small, single-purpose abstractions — TravelMethod , Breakable , DestinationList , and Area with its TileCondition collaborators — each introduced because REQ2 itself already repeats the behaviour it captures across at least two of its own classes, not in anticipation of a requirement that has not been written. The costs were accepted deliberately and are listed rather than hidden: a reader needs to learn a handful of small types before any one mechanism's code is self explanatory, BreakableGround 's two template hooks are slightly wider (protected) than today's single-package use needs, and Puddle 's two-state condition would need to become a class hierarchy if a third charge level were ever specified. None of these affect the game's behaviour; they are the kind of compromise the brief's own "note on design" sections anticipate, traded for a dispatcher (TravelAction , BreakAction) that has not needed to change once since the first of its three concrete mechanisms, respectively three concrete breakables, was added.

Poyee Tan 36641162 – REQ 3: The Merchant

This section explains how we designed REQ3, where the Farmer buys from and sells to a wandering Merchant. Each part compares several designs. Each comparison checks SOLID, DRY and connascence. Section J explains every use of final, protected and static. Section K lists what changed since Assignment 1.

A. Buying and Selling Abstractions (Purchasable and Sellable)

The spec says that "later versions of the game may add more things to buy and sell". So the first question is how the Merchant knows what it can trade. The answer shapes every later part.

Design 1: Merchant handles every product with instanceof / if-else

Pros	Cons
All trading rules sit in one class.	Merchant would hold the coin rules and the effects of seven products (violates SRP).
No extra interfaces are needed.	Every new product changes Merchant (violates OCP).
	Merchant depends on every concrete item. This is Connascence of Type, and it grows with each product (violates DIP).
	Every branch repeats the code that pays the seller (violates DRY).

Design 2: Purchasable and Sellable interfaces with polymorphism (Current)

Pros	Cons
Each product owns its price and its effect. Merchant only lists actions (SRP).	The design has more classes to read.
Merchant finds sellable items with item.asCapability(Sellable.class). So it never names a concrete item (DIP, engine integration).	A reader must open several files to follow one trade.

Pros	Cons
A new product is a new class. Merchant, BuyAction and SellAction do not change (OCP).	
Purchasable has three methods and Sellable has two. So each product only depends on what it uses (ISP).	
Callers only know method names. This is Connascence of Name, the weakest kind.	

Final Design & Justification: We chose Design 2. BuyAction and SellAction only hold a Purchasable or a Sellable. Java then runs the right purchase() or sell() for each product. So no class needs a type check, and the payment code is not repeated (DRY).

Package placement (change during A2): Purchasable, Sellable and MerchantContext moved from game.trading to game.actions. Before the move, the packages formed a loop: actions used trading, trading used items, and items used actions. Now the products depend on game.actions, and game.actions depends on none of them. The loop is gone, and the actions own the interfaces they need (DIP).

Maintainability Foresight: To sell copper ore, we add one class that extends SellableItem. To offer a new crop, we add one CropPurchase subclass to the catalog. No trading class changes.

B. Passing Trade Information (MerchantContext)

Many effects happen next to the Merchant. A crop is planted there, and a chicken or a rock may appear there. So each product needs the Merchant and its location.

Design 1: Pass the Merchant and its Location as two separate parameters (previous version)

Pros	Cons
The code is simple and needs no extra class.	The same two parameters appeared in about ten method headers (violates DRY).
	Every caller had to pass them in the right order. This is Connascence of Position.
	A third piece of information would change every header (violates OCP).

Design 2: Pass the whole GameMap and let each product find the Merchant

Pros	Cons
Each product can reach everything it might need.	Every product would repeat the search for the Merchant (violates DRY).
	Each product would depend on much more than it uses (violates ISP).

Design 3: A small MerchantContext object (Current)

Pros	Cons
The pair is defined in one place (DRY).	The design adds one more class.

Pros	Cons
Callers use getMerchant() and getLocation(). So Connascence of Position becomes Connascence of Name.	An old context would hold an old location. Merchant avoids this by creating a new context every turn.
New information only needs one new getter. No method header changes (OCP).	
The class only carries data, and its fields are private final (SRP, encapsulation).	

Final Design & Justification: We chose Design 3. It removes the repeated parameters and weakens the connascence from Position to Name. It is also much simpler than passing the whole map (KISS).

C. Handling Sellable Items (SellableItem)

All four sellable items follow the same steps. First, the game checks whether the Merchant pays. Next, the seller gets the coins. Finally, the item runs its own effect.

Design 1: Independent implementations

Pros	Cons
Each item is easy to write at first.	Diamond, Egg, Milk and Stone each repeated the payment and the message (violates DRY).
	All four classes had to follow the same steps. This is Connascence of Algorithm across four files.
	One item could forget to pay, so SellAction could not trust every Sellable (risks LSP).

Design 2: Composition via a shared helper method

Pros	Cons
The helper removes most of the repeated code.	Each item must still call the steps in the right order. This is Connascence of Execution.
Items keep their single parent class free.	A developer could still pay the seller after the Merchant refuses.

Design 3: Abstract base class SellableItem with the Template Method pattern (Current)

Pros	Cons
The payment and the message exist once in SellableItem (DRY).	Each item uses up its single parent class.
The final sell() method fixes the order of the steps in one place. So the Connascence of Execution stays local.	A reader must open two files to see a full sale.
Subclasses only override merchantRefusesPayment() or applySaleEffect(). So each subclass only describes how it is different (SRP, OCP).	The hooks have defaults, so a missing override does not cause a compile error.
No subclass can skip a step, so every SellableItem	

Pros	Cons
works in SellAction (LSP).	

Final Design & Justification: We chose Design 3. SellableItem.sell() calls the two hooks, and Java runs the version in Milk, Diamond, Egg or Stone. Milk only overrides the refusal. Egg, Stone and Diamond only override the effect. Using up the parent class is acceptable, because none of our items needs another parent (KISS).

Use of final and protected here: sell() is **final** and the hooks are **protected**. Section J explains why.

Maintainability Foresight: CopperOre only needs to extend SellableItem and override one hook if needed. Merchant and SellAction do not change (OCP).

D. Sale Effects that Create Objects (Egg and Stone factories)

Selling an Egg may make a Chicken appear. Selling a Stone builds a Rock. A team member changed this part during A2.

Design 1: Egg creates new Chicken() and Stone creates new Rock() directly (previous version)

Pros	Cons
The effect is easy to see inside the item.	Egg depended on Chicken, and Stone depended on Rock. Rock already creates Stone, so this formed a loop.
	Each item was tied to one concrete class. This is Connascence of Type (violates DIP and OCP).

Design 2: A separate factory interface for each effect

Pros	Cons
The items are fully separated from what they create.	Two new interfaces only wrap one line of code each (violates KISS).

Design 3: Inject a Supplier<Actor> / Supplier<Ground> through the constructor (Current)

Pros	Cons
Egg and Stone only depend on the engine types Actor and Ground. So the loop is gone (DIP).	Every place that creates an Egg or a Stone must pass a factory.
The creator decides what appears. Chicken passes a Chicken factory, and Rock passes Rock::new (SRP).	A reader cannot see from Egg alone which animal appears.
One Egg class works for any animal, so we need no ChickenEgg or DuckEgg copies (DRY, OCP).	
Egg only calls get(). This is Connascence of Name.	

Final Design & Justification: We chose Design 3. It removes the loop with the smallest change. It also uses the standard Supplier type, so it adds no new interface (KISS). The factories are private final, because they never change after construction.

E. Buying Crops and the Sword (CropPurchase)

Parsnip and Tomato purchases follow the same steps. The game finds an empty tile next to the Merchant and plants the crop. Then the buyer pays, unless the payment is waived. Finally, the crop runs its extra effect. The Sword plants nothing.

Design 1: ParsnipPurchase and TomatoPurchase each write the whole purchase

Pros	Cons
Each class is complete on its own.	The tile search, the planting and the payment would be copied (violates DRY).
	Both classes must follow the same steps. This is Connascence of Algorithm.
	A fix to the planting rule would be needed in every class.

Design 2: One CropPurchase class with flags (price, crop, waiver chance, drop count)

Pros	Cons
Only one class is needed.	Different effects become flags and numbers. This is Connascence of Meaning.
	Every new kind of effect adds another flag and branch (violates OCP and SRP).

Design 3: Abstract CropPurchase with a final template method (Current)

Pros	Cons
purchase() is written once and is final. So no subclass can change the order of the steps (DRY, LSP).	Each crop purchase uses up its single parent class.
Subclasses only fill in createCrop() (a Factory Method), applyPlantingEffect() and, optionally, rollPaymentWaiver() (SRP, OCP).	A reader must open two files to see a full purchase.
The "empty tile" rule is the Unplanted condition, which Cauliflower also uses. So the rule exists once (DRY).	Readers see two styles: a template class and a direct implementation.
SwordPurchase implements Purchasable directly. So it does not inherit planting code it cannot use (LSP, ISP).	

Final Design & Justification: We chose Design 3. It keeps the planting rules in one place and gives each crop a small class. The connascence left is Connascence of Name on the hooks. When there is no empty tile, the method simply does not charge. This is simpler than charging first and refunding later (KISS).

Use of final and protected here: purchase() is **final** and the hooks are **protected**. createCrop() and applyPlantingEffect() are abstract, because every crop needs them. rollPaymentWaiver() has a default, because only Tomato needs it.

Maintainability Foresight: A Melon purchase only needs a MelonPurchase subclass with two short methods.

F. The Merchant's Catalog (constructor injection)

Design 1: A default constructor that creates the three purchases

Pros	Cons
The Merchant is easy to create with new Merchant().	Merchant always depends on three concrete purchases (violates DIP; Connascence of Type with a degree of 3).
	A different stock means changing Merchant (violates OCP).

Design 2: A MerchantFactory class

Pros	Cons
The factory removes the dependencies from Merchant.	We only create one Merchant, so a factory class is unnecessary (violates KISS).

Design 3: Inject the catalog through a varargs constructor (Current)

Pros	Cons
Merchant only depends on Purchasable (DIP). Its degree of connascence drops from 3 to 1.	The caller must build the catalog.
StackQueueValleyWorld already builds every object, so it also decides the stock (SRP).	The degree of 3 moves to StackQueueValleyWorld.
Different Merchants can sell different things without changing Merchant (OCP).	
The Merchant keeps its own copy of the array. So the caller cannot change the stock later (encapsulation; no Connascence of Identity).	

Final Design & Justification: We chose Design 3. StackQueueValleyWorld already depends on every concrete class, so moving the three purchases there adds no new coupling. Merchant only knows the interface, and no product is listed twice (DRY).

G. Random Chances (RandomUtils)

Every 50% chance in REQ3 uses RandomUtils. This covers the Tomato waiver, the Sword failure, the Milk refusal, the Diamond theft and the Egg hatching. The REQ1 rationale explains this class in full, so this part is short.

Design 1: Each class creates its own new Random() (original version)

Pros	Cons
Each class works on its own.	About 20 classes repeated random.nextInt(100) < X (violates DRY).

Pros	Cons
	All of them had to use the same formula. This is Connascence of Algorithm. One typo would change a 50% chance.

Design 2: Inject a random object into every class

Pros	Cons
Each class depends on an abstraction (DIP).	About 20 constructors and their callers would change (violates KISS).

Design 3: A static facade over a replaceable instance (Current)

Pros	Cons
Callers keep simple static calls, so no constructor changes (KISS).	All classes share one changeable instance (Connascence of Identity).
Each static method passes the call to one instance. A test can replace it with setInstance() (OCP).	A class's constructor does not show this dependency.
doRollPercent() is built on doRandomInt(). So one override controls every roll (DRY).	
The formula lives in one class. So Connascence of Algorithm becomes Connascence of Name.	

Final Design & Justification: We chose Design 3. It keeps the simple calls and still lets tests control the chances. The shared instance is acceptable, because the game runs on one thread.

H. Coins (FarmerStatistics, BaseStatistic and StatisticAdjuster)

Design 1: An int coins field in Farmer

Pros	Cons
The field is very easy to read.	We would rewrite the 10,000 cap that the engine already provides (violates DRY).
	Every class that pays or charges would need to know Farmer (violates DIP).

Design 2: An engine statistic plus a static helper (previous version)

Pros	Cons
The engine's BaseStatistic caps coins at 10,000.	Every call repeated the actor as the first argument (Connascence of Position).
One hasEnough() check serves coins and stamina (DRY).	Static helpers cannot be replaced or extended.

Design 3: An engine statistic plus an instance-based StatisticAdjuster (Current)

Pros	Cons
Coins use the FarmerStatistics.COINS enum key. So the cap comes from the engine (KISS).	Each check creates a small object.
An enum key gives Connascence of Name that the compiler checks.	
StatisticAdjuster takes the actor once. Then hasEnough(), trySpend() and increase() work for any statistic (DRY).	
StatisticAdjuster only does the maths. StatisticChange builds the message (SRP).	
BuyAction checks coins with the same hasEnough() that stamina uses.	

Final Design & Justification: We chose Design 3. It reuses the engine's cap and removes the repeated actor argument. BuyAction checks the price before calling purchase(), so no product repeats the check.

I. The Merchant Wanders (WanderingActor and WanderBehaviour)

The spec says the Merchant wanders around the map, like the Chicken and the Cow.

Design 1: Merchant extends Animal to reuse its wandering

Pros	Cons
No new class is needed.	Animal implements Feedable, so the Merchant could be fed (violates LSP).

Design 2: WanderingActor with the movement written inside playTurn() (intermediate version)

Pros	Cons
Merchant, Chicken and Cow share the movement (DRY).	The movement is fixed inside the actor. A new movement style needs a new subclass (violates OCP).

Design 3: WanderingActor uses an engine Behaviour, WanderBehaviour by default (Current)

Pros	Cons
The engine's Behaviour decides what an NPC does. This is how the engine expects NPCs to work.	The design adds one more class.
WanderingActor depends on Behaviour, not on one movement (DIP). A different behaviour can be passed in (OCP).	The behaviour is private final, so an actor cannot switch it during the game.
Any actor can reuse WanderBehaviour, including the crow's Perching state (DRY).	

Final Design & Justification: We chose Design 3. Merchant, Chicken and Cow share WanderingActor, and the movement itself comes from an engine Behaviour. So a new movement style is a new Behaviour, not a new subclass (OCP, DIP). Unlike Design 1, the Merchant does not inherit Feedable, so it can never be fed (LSP).

Shared unconscious check (change during A2): Farmer, WanderingActor and Crow each repeated the same "fall unconscious and do nothing" check (Connascence of Algorithm). Now they extend ConsciousActor. Its playTurn() runs the check once and then calls the protected hook playConsciousTurn() (DRY).

J. Use of final, protected and static

Each of these keywords limits how a class can be extended. So we kept each use only when its benefit is bigger than its cost. No REQ3 class is marked final, so every class can still have subclasses.

Keyword and location	Why it is used here	Pros	Cons
final methods: SellableItem.sell(), CropPurchase.purchase()	They hold the steps that SellAction and BuyAction trust.	Payment always follows the refusal check. A crop is never charged without room (LSP).	An item with different steps cannot override them. It must implement the interface directly, as SwordPurchase does. The benefit is bigger.
final methods: getSellPrice(), getPrice(), getProductName()	The menu and the payment read the same private field.	The shown price is always the charged price (Connascence of Value in one place).	Prices cannot change during the game. We do not need that yet.
private final fields: catalog, sellPrice, price, productName, MerchantContext fields, factories, action fields, behaviour	The constructor sets each value once.	No class can change a price or a product during a trade.	A change needs a new object. Private fields do not limit subclasses.
private static final constants: PRICE, SELL_PRICE, *_CHANCE_PERCENT, PARSNIPS_PRODUCED, MAX_HP, STARTING_COINS	Each number from the spec gets a name.	No magic numbers. Connascence of Meaning becomes Connascence of Name (DRY).	Other classes cannot reuse them. A value that varies goes into the constructor instead.
protected constructors: SellableItem, CropPurchase	Both classes are abstract bases.	Only subclasses can create them.	No real cost.
protected hooks: merchantRefusesPayment(), applySaleEffect(), createCrop(), applyPlantingEffect(), rollPaymentWaiver(), playConsciousTurn()	Subclasses change one step of a template.	Callers can only use the public method, so no step is skipped. Fields stay private.	Classes in the same package can also see them. A default hook can be forgotten silently.
playTurn() in ConsciousActor is not final	A future actor may need to replace the whole	The class stays open for extension (OCP).	An override could skip the unconscious check.

Keyword and location	Why it is used here	Pros	Cons
	turn.		
static: RandomUtils methods and instance field	About 20 classes use them.	Call sites stay simple, and tests can swap the instance.	The instance is shared, changeable global state.
static factory methods: Area.adjacentTo(), ItemDropper.adjacentTo()	They name common tile searches shared with REQ1 and REQ2.	Call sites are easy to read, and the tile rules exist once (DRY).	Static methods cannot be overridden.

K. Changes Since Assignment 1

Class	A1 design	A2 change	Reason
Egg, Milk	Plain items.	Extend SellableItem. Egg takes a Supplier<Actor>.	REQ3 sells them. The Supplier removes Egg's link to Chicken (DIP).
Stone, Diamond	Plain items.	Extend SellableItem. Stone takes a Supplier<Ground>.	They share the selling steps (DRY). The Supplier removes the Rock-Stone loop.
Animal	Extended Actor with its own movement.	Extends WanderingActor.	Merchant also wanders, so the movement is shared (DRY) without making it feedable (LSP).
Farmer	Only health and stamina.	Adds COINS and extends ConsciousActor.	The engine caps coins. The unconscious check is shared (DRY).
StackQueueValleyWorld	Placed A1 actors.	Builds the Merchant with its catalog.	It decides the stock, so Merchant only knows Purchasable (DIP).

REQ 5: Stateful Creature, The Crow

REQ 5 asks for one stateful creature with exactly four distinct states, at least two of which exhibit genuinely new behaviour. We implemented a *Crow* that cycles between four states — *Perching*, *Foraging*, *Mobbing* and *Fleeing* — of which Foraging, Mobbing and Fleeing are new behaviours.

All four states share one framework. Rather than storing an enum on the Crow and branching on it with a switch inside playTurn() (which would pile every state's behaviour and transition rule into one low-cohesion class and force edits to that switch for every new state), each state is its own class implementing CreatureState (act, nextState, onEnter). A single generic CreatureStateAction asks the current state for nextState(); if it differs, it switches state and runs the new state's onEnter(), otherwise it runs act(). The states are stateless and shared as one INSTANCE each; any data that differs between crows lives on the Crow behind small capability interfaces. The sections below give the key design decision for each of the four states.

A. Perching (default idle state)

Design 1: Implement Perching's entry effect (the crow's droppings freely water nearby crops) by importing Crop and calling its watering methods directly on every crop on the surrounding tiles, and write the random hop for its ordinary turn inside Perching itself.

Pros	Cons
● Direct and obvious; introduces no new abstraction. ● All of the state's logic sits in one class.	● game.crow gains a compile-time dependency on game.crops, tightly coupling two otherwise unrelated features. ● Perching must know Crop's concrete API, and supporting any other freely-waterable ground would mean editing Perching. ● The random-hop logic would duplicate the wandering already written for the animals and the Merchant (violates DRY).

Design 2: Depend on abstractions instead. Perching asks each tile within 1 tile for a FreelyWaterable via location.getGroundAs(...) and calls receiveFreeWatering(); it never references Crop. Its ordinary turn is delegated to an injected Behaviour, by default the existing WanderBehaviour.

Pros	Cons
● game.crow has no compile-time dependency on game.crops, keeping the two packages decoupled (Dependency Inversion Principle). ● Any ground that implements FreelyWaterable is watered automatically, with no change to Perching (Open-Closed Principle). ● Reuses WanderBehaviour, so the "take one random step" rule is written once (DRY), and a different idle behaviour can be injected through the constructor.	● One layer of indirection: the concrete watering effect lives in whatever implements FreelyWaterable rather than in Perching itself.

The final design uses Design 2. Routing the watering effect through the FreelyWaterable capability lets a crow state reach into the REQ 1 crop feature without either package depending on the other, and delegating the idle hop to WanderBehaviour keeps Perching focused on its own transitions (to Mobbing for a lone target carrying items, to Foraging when food is in range).

B. Foraging (new behaviour)

Design 1: Let the Foraging state object remember which items this crow has already hidden by keeping a Set<Item> field on Foraging itself, so each Crow needs its own Foraging instance.

Pros	Cons
● Intuitive: the state that uses the data also holds it. ● No need to expose any per-creature data through the Crow.	● Foraging can no longer be shared between crows; every crow needs its own full set of state objects. ● Fragile: if a Foraging instance were ever shared by accident, one crow's hidden items would silently leak into another crow's memory. ● The "is this item worth foraging?" rule would be needed both by Foraging and by the transition checks in Perching and Fleeing, inviting duplicated checks.

Design 2: Keep Foraging stateless and store the remembered items on the Crow behind a narrow ItemCache capability (remembers / remember). The rule "an item worth foraging is one this crow has not cached" is written once in ForageableItems and used by both CrowSensing (for transitions) and Foraging (for caching).

Pros	Cons
● One shared Foraging.INSTANCE serves every crow safely, because the state holds no per-creature data. ● Foraging depends only on the small ItemCache interface rather than on Crow (Interface Segregation and Dependency Inversion Principles). ● The forageable-item rule lives in one place (DRY), so the crow never re-forages its own cache whichever state is asking.	● Adds an extra interface (ItemCache) and a helper class (ForageableItems). ● State classes must be kept stateless as a discipline.

The final design uses Design 2. Stateless Foraging plus the ItemCache capability lets every crow share one state object while each keeps its own memory of hidden items. Its entry effect (scattering the items around the food it swoops on) and its caching behaviour stay inside Foraging, while the condition that triggers it is shared through CrowSensing.

C. Mobbing (new behaviour)

Design 1: Write Mobbing's own logic for finding its target (scanning nearby tiles for a lone actor carrying items) and for knocking the target's items loose (looping over the inventory and dropping each item on a random adjacent tile) directly inside Mobbing.

Pros	Cons
● Self-contained: everything Mobbing does is visible in one class. ● No helper classes to learn.	● The same neighbour-scanning is needed by Perching (to decide when to start mobbing) and by Fleeing, so it would be copied across states (violates DRY). ● The "drop everything onto random adjacent tiles" code would be duplicated in Fleeing, which drops its carried items in the same way. ● Mobbing would mix sensing, item placement and state behaviour, lowering cohesion (violates SRP).

Design 2: Reuse shared helpers. CrowSensing answers hasLoneTargetWithItems(), nearestActor(), isCrowded() and isSafe(); ItemDropper.dropAllCarriedBy() performs the knock-loose drop; CrowNavigation.stepToward() moves the crow. Mobbing itself only decides what to do with those answers: dive on entry, step closer, then steal one random item. The crow eats the item at once if it is food it can eat (Consumable.canBeConsumedBy), and otherwise keeps it in its own inventory.

Pros	Cons
● Mobbing stays focused on its own behaviour (Single Responsibility Principle); sensing and dropping are each written once (DRY). ● Its nextState() reads as a short expression: crowded → Fleeing, nobody nearby → Perching, otherwise stay. ● Stolen non-food items sit in the crow's normal inventory, so Fleeing's existing drop effect returns them to the map with no Mobbing-specific code.	● Behaviour is spread across several small classes, so reading Mobbing means following calls into CrowSensing and ItemDropper. ● Changing a shared helper affects every state that uses it.

The final design uses Design 2. Delegating sensing, movement and dropping to shared helpers keeps Mobbing small and lets it cooperate with Perching and Fleeing without duplicated code, while still delivering clearly new behaviour (diving, chasing and stealing).

D. Fleeing (new behaviour)

Design 1: Track how many consecutive turns the crow has felt safe with an int counter field inside the Fleeing state object, and write the scanning for nearby actors and the dropping of carried items with ad-hoc loops inside Fleeing.

Pros	Cons
● Simple: the counter lives exactly where it is used. ● No new interface is needed.	● Fleeing could not be shared between crows; a shared instance would let one crow's calm-down progress leak into another's. ● Resetting the counter when re-entering Fleeing depends on remembering to clear the field, which is easy to get wrong with per-instance state. ● Duplicates the scanning and dropping logic already needed by other states (violates DRY).

Design 2: Keep Fleeing stateless and store the safe-turn counter on the Crow behind a narrow Composure capability (getSafeTurns, recordSafeTurn, resetComposure). onEnter() resets composure, drops everything the crow carries via ItemDropper and startles adjacent actors one step back via CrowNavigation.stepAway(); nextState() returns to Perching after 3 consecutive safe turns, or to Foraging if safe with food in range.

Pros	Cons
● One shared Fleeing.INSTANCE works for every crow, and Fleeing sees only Composure rather than the whole Crow (Interface Segregation Principle). ● The calm-down rule is explicit: reset on entry, increment while safe, reset when threatened. ● Reuses ItemDropper and CrowNavigation, which Mobbing and Foraging also use (DRY).	● Another small interface (Composure) has to be defined and implemented by Crow. ● The counter lives away from the state that interprets it, so readers must look in two places.

The final design uses Design 2. Moving the counter onto the Crow behind Composure keeps Fleeing stateless and shareable while making its calm-down behaviour reliable, and it closes the cycle of the state machine by handing control back to Perching or Foraging.

E. KISS, connascence and foresight

KISS. The crow's turn is one line: *playConsciousTurn()* returns a *CreatureStateAction*. The protocol has two plain decisions: is the next state different, and which method do we call? The transition graph is four short lists of conditions, checked in order. It has no scores, no weights and no random choice.

Connascence. A *switch* on an enum would tie the enum to every *switch* by position (Connascence of Position). Now the action and a state share only an interface (Connascence of Type). The order "*nextState*, then *onEnter* or *act*" is Connascence of Execution, and we keep it in one place, *CreatureStateAction*. The sensing radius and the crowd size were once repeated in several places (Connascence of Value and of Algorithm). Now each is one constant, and "crowded" and "safe" are defined once in *CrowSensing*. If the states kept the hidden items, two crows would share them (Connascence of Identity). Now each crow owns its data behind a capability interface, and we compare states with *equals*, so a new instance of a state never re-runs its entry effect.

Foresight. A fifth crow state is one new class plus one condition in the states that lead to it. A second crow needs no new state objects. Letting the crow smell food from three tiles is one edit of *SENSING_RADIUS*. To log every transition, or to let a creature act on the turn it changes state, we change only *CreatureStateAction*. The chameleon in the brief would need its own state classes and *StatefulCreature*, and nothing else. A second creature that retreats from the player needs *stepAway* made public, or a state that calls *Area.farthestFrom*.

F. Changes to the Assignment 1 design

REQ5 changes the Assignment 1 design in four places. Each change has a reason, and none of them changes what the Assignment 1 classes did before.

Change from Assignment 1	Why we made it
Consumable gained a default method *canBeConsumedBy(actor)*, and *ProcessedProduct* gained *canReceiveProcessingEffect(actor)*.	The crow eats food it steals, but it has no stamina. A dried product restores stamina, so the crow must refuse it. The defaults allow every actor. So the Assignment 1 Decorator and every other product work as before. Only *DriedProduct* overrides the hook.
Animal and *Farmer* extended *Actor*. *Animal* now extends *WanderingActor*, which holds a *Behaviour*. Both now extend *ConsciousActor*.	The crow's Perching state needs the same random step as the animals. The Farmer, the animals and the crow need the same unconscious check. A shared base class and one *WanderBehaviour* remove the copies.
FreelyWaterable is a new capability on *Crop*.	The crow's droppings water crops without a tool. A small new interface lets *game.crow* water them without importing *game.crops* (DIP). *Waterable* does another job.
The display characters of the Cow, Chicken and Milk changed (C to B, c to n, M to K).	REQ1's rationale gives the reason. The crow (*v*) and the items also stay easy to read on the map.

G. Known minor violations and why we accept them

Violation or cost	Why it is acceptable
The transitions live in the four states, not in one table. OCP is only partly met, because a new state edits the states that lead to it.	Each state's conditions sit next to its behaviour, and the driver does not need to know the graph. A central table would need the conditions passed in as objects. That would move each condition away from its behaviour.
Fleeing.nextState() changes the calm-turn counter. So for this state it is not a pure question.	The counter must change every turn, including the turn the crow decides to leave. *nextState()* is the only method that runs on every turn, and the action calls it exactly once per turn.
Each state has a public static final INSTANCE. We also use *static final* constants such as *CALM_DOWN_TURNS*, *CROWD_SIZE* and *SENSING_RADIUS*.	The states hold no data about one creature, so one instance is enough. They stay non-*final*, have public constructors and are equal by type. REQ5 has no *protected* member and no mutable static state.
We use the engine's Behaviour for wandering, not for the states.	*WanderBehaviour* is shared by the Chicken, Cow, Merchant and Perching, so the rule exists once (DRY). A *Behaviour* returns an action and knows nothing about the crow's state. So it cannot own the transition graph or the entry effect that REQ5 requires. The other states' steps depend on what the crow senses, so they stay in *CrowNavigation*.
Messages describe what the crow touched and say when there was nothing to act on.	The messages are longer. In return, we can check each step from the console.

Summary

Across the four states the design keeps responsibilities small and dependencies pointed at abstractions. Each CreatureState owns exactly one state (SRP); a fifth state can be added as a new class without editing CreatureStateAction or Crow (OCP); any CreatureState substitutes for any other wherever CreatureStateAction uses one, all honouring the same three-method contract (LSP); the capability interfaces StatefulCreature, ItemCache and Composure are kept separate and minimal, so Foraging sees only ItemCache and Fleeing sees only Composure (ISP); and the driver and Perching's watering effect depend only on abstractions, never on Crow or Crop by name (DIP). Shared logic is written once (DRY): CrowSensing centralises all neighbour-scanning for every state's nextState(), ItemDropper.dropAllCarriedBy backs both Mobbing's and Fleeing's drop effects, CrowNavigation provides stepping toward and away for Foraging, Mobbing and Fleeing, and WanderBehaviour is reused by the animals, the Merchant and Perching.